Training Small GPT Models on TinyStories: Model Size, Data, Context, and Fixed-Time Evaluation

Steffany Lara (A0350565H) Kerem Ozemre (A0350378Y) Tonio Kowalke (A0350429E) Ahmed Haj Hassine (A0342398U)

National University of Singapore

DSA4212: Optimisation for Large-Scale Data-Driven Inference

November 2026

Under a short training-time budget, increasing model capacity reduces the amount of text a model can process. We investigate this trade-off by training GPT models from scratch on TinyStories. An initial 20-run screen crosses five model families (16–253 million parameters) with four training-data fractions under a one-hour budget. We report the screen, token exposure, historical test scores, generated examples, and four budget/context extensions. The screen favours the 124M family, but unequal validation samples and a single seed motivate a common-validation audit and six fresh XL/XXL trials. Across three seed pairs, XL achieves lower reported validation cross-entropy: mean 1.4947 versus 1.5244, with approximately 2.9% lower geometric-mean perplexity and 1.93 times the training exposure. This supports an observed advantage under the tested recipe and recorded H100 MIG allocation. Limited replication and changed conditions in supplementary runs constrain broader conclusions. A controlled equal-target comparison confirms this exposure reading: at 143M matched targets XXL leads XL by 0.053 nats (CE 1.5209 versus 1.5743), reversing the one-hour ranking.

1 Introduction

Declarations. *Compute budget:* one hour (3,600 s) of in-loop wall-clock per run on one NVIDIA H100 NVL MIG 3g.47gb slice in bfloat16, fixed before the final runs; data preparation, final validation, and checkpointing are excluded. *Initialisation:* random weights (nanoGPT initialisation); no pretrained weights, embeddings, or tokenizer fine-tuning. *Evaluation objective:* held-out cross-entropy (nats per target), also reported as perplexity and bits per byte. *Code and checkpoints:* https://github.com/KeremOzemre/Optimization-for-Large-Scale-Data and https://huggingface.co/dsa4212-tinystories/tinystories-nanogpt.

A larger language model can represent more complicated patterns, but each update costs more computation. Under a fixed training clock, extra capacity therefore comes with fewer updates and less text exposure. Our practical question is: *which tested GPT family predicts held-out TinyStories text best after one hour under a shared training recipe and recorded GPU allocation?*

TinyStories is a synthetic collection of simple English stories intended to make coherent generation accessible to small models [1]. We use this restricted domain for a manageable from-scratch study. Research on compute allocation motivates examining model size and processed tokens together: Kaplan et al. [6] favour a large model stopped early, while Hoffmann et al. [5] balance size and tokens near 20 per parameter. Closer to our budget, Geiping and Goldstein [2] train a model in one GPU-day, and a five-minute laptop study on TinyStories [3] reports a best size near 2M parameters. Our controlled resource is wall-clock time, rather than measured floating-point operations.

We first screen five model sizes and four data fractions. Loss improves strongly through XL, while XXL offers no recorded one-hour improvement and data-fraction effects are small. Token audits, shorter-budget and longer-context trials, test scores, and generated stories help interpret that pattern. A close XL/XXL gap, unequal validation workloads, and only one seed then motivate common scoring and fresh repetitions. This paper includes all 33 completed research runs and the four-checkpoint audit; an appendix retains the nine pilot/development records. Copies of the original logs in two directories count once.

2 Methods

2.1 Data and token sampling

We load `roneneldan/TinyStories` with Hugging Face `datasets`. The official training split contains 2,119,719 stories. We divide the official validation records 80/20 with preparation seed 42, obtaining 17,592 validation and 4,398 test stories. These are disjoint records; duplicate or paraphrased content has not been audited.

The fixed GPT-2 tokenizer from `tiktoken` has 50,257 entries. We append end-of-text after each story and cache concatenated token IDs as `uint16` streams. Full training, validation, and test caches contain 473,992,236, 3,808,644, and 957,274 tokens. The 25/50/75% story

subsets contain approximately 118.45/236.97/355.37 million training tokens. Fractions describe available stories, not an imposed number of processed tokens.

Training samples uniform token-window starts with replacement. Inputs and next-token targets are shifted by one position. Causal attention restricts each prediction to its current and preceding inputs. Windows can cross story boundaries; end-of-text does not reset attention. Consequently, processed *targets* count repeated predictions. Dividing them by cache length gives cache-equivalent exposure, without proving a complete epoch or unique coverage.

2.2 Model and training recipe

We reuse nanoGPT's decoder-only Transformer [7, 11] at commit `3adf61e154c3`, retaining its MIT attribution, random initialization, causal attention, learned position embeddings, and tied input/output weights. No pretrained weights are loaded. Table 1 lists the families. Depth, width, and heads change together, so their individual effects are not isolated. Width is divisible by the head count. Core runs use context 256 and dropout 0.1.

Training minimizes average next-token cross-entropy using PyTorch automatic differentiation and AdamW [9]. Shared settings are peak learning rate 3×10^{-4}, weight decay 0.1, betas (0.9, 0.95), and gradient-norm clipping at 1.0 after accumulation. Different microbatches preserve 16 sequences, or 4,096 targets, per optimizer update. CUDA bfloat16 autocasting is used with float32 stored parameters, on one device without compilation or distributed training. The shared recipe is not tuned separately for each family.

The learning rate rises from 3×10^{-5} to its peak during the first 2% of the time budget, then follows cosine decay back to the starting rate. This budget-aware schedule [8] keeps fast and slow runs at the same point of the schedule and avoids penalising a run cut mid-decay [4]. Warmup lasts 72 seconds in one-hour runs and 36 seconds in half-hour runs. The timer excludes preparation and model/optimizer initialization, includes periodic validation and logging, and stops after the current update. Final validation and saving occur afterward. We retain the final checkpoint, without early stopping or selecting an intermediate iterate.

The screen and repetitions record *NVIDIA H100 NVL MIG 3g.47gb*, totalling 26 nominal slice-hours; the matched-target pair and the reduced-rate run add about 2.5 hours on the same slice. The four extensions add 2.5 nominal hours labelled *NVIDIA H100 NVL*; identical allocation is unverified. Offline evaluation and development costs are additional. Matching hardware labels do not guarantee identical live load.

2.3 Assessment and analysis

Evaluation disables dropout and gradients. We report token-weighted cross-entropy (CE, nats per target) and perplexity (PPL), its exponential. Original validation uses 16 fixed microbatches: 65,536 targets for Small/Medium/Large, 32,768 for XL, and 16,384 for XXL. Fresh repetitions instead use 65,536 targets for both families, evaluation seed 43, and approximately five-minute intervals. Preparation seed 42 is independent of training seed.

The common offline scorer traverses non-overlapping 256-token windows, resets context each window, includes the tail, and defaults to float32. Returned rows report 3,808,643 validation targets, matching the intended count. The scoring options of each audit are saved in `analysis/followups/*/protocol.json`. Across training seeds we report individual values, mean CE, sample standard deviation, and geometric-mean PPL.

Historical full-test scoring uses float32 and non-overlapping 256-token windows but drops the short tail, scoring 957,184 targets. Bits per byte (BPB) divides negative log-likelihood in bits by decoded target-token byte lengths, including end-of-text representations. Its external comparability therefore depends on special-token and context policy. The four extension test exports are sampled, with unequal counts and incomplete protocol metadata; they are separate from the full-test endpoint.

Each original model also has five fixed-prompt continuations: temperature 0.8, top-k 40, maximum 150 new tokens, and seed 42 reset for each prompt. The legacy export truncates displayed text at the first new end-of-text token. These 100 outputs were not blindly rated. We additionally audit target counters, calculate measured throughput and cache-equivalent exposure, and use $6ND$ as an approximate training-compute proxy, where N is parameters and D is processed targets. This proxy does not measure GPU utilization.

3 Experimental sequence

The original 5×4 screen combines the five families with 25/50/75/100% of training stories. All runs use seed 42 and a one-hour budget. This explores capacity and available-data effects with one observation per cell.

To explore budget sensitivity, Large, XL, and XXL are trained afresh on full data for 1,800 seconds, keeping their original batches and seed 42; validation is every 150 seconds. We compare them with the original full-data one-hour runs. A separate XL run extends context to 512 for one hour. Microbatch 4 and accumulation 4 preserve 16 sequences but double targets/update to 8,192. Position embeddings add 196,608 parameters (total 123,944,448). Validation still scores 32,768 targets, with different windows/context. Hardware labels differ from the screen, and each extension has only one seed.

Table 1: Architecture families. Parameter counts include learned position embeddings and count the tied token/output matrix once. All use context 256, dropout 0.1, and an effective batch of 16 sequences.

Family	Layers L	Width d	Heads H	Parameters	Microbatch × accumulation
Small	4	256	4	16,079,360	16×1
Medium	6	384	6	30,018,816	16×1
Large	8	512	8	51,037,184	16×1
XL	12	768	12	123,747,840	8×2
XXL	16	1024	16	253,085,696	4×4

The later audit re-scores Large/full, the original validation-selected XL/50%, and XL/full and XXL/full on common validation. This checks measurement rather than retraining variability. We then train fresh XL and XXL models with seeds 42/43/44, keeping the full cache, preparation seed, evaluation seed, optimizer, schedule, and corrected validation workload fixed. Fresh seed-42 runs avoid mixing old assessment overheads and cache provenance with the new trials. Finalists were selected after inspecting the screen; the follow-up is targeted replication of an exploratory finding.

4 Results

4.1 Initial size and data-fraction screen

Figure 1 shows the original experiments; Table 6 retains all 20 validation and test endpoints. Full-data sampled CE falls from 1.8386 (Small) to 1.6444 (Medium), 1.5460 (Large), and 1.5068 (XL), then rises to 1.5106 (XXL). XL/50% has the smallest sampled score, 1.5025. These broad patterns favour XL, while unequal validation samples complicate the close finalist comparison.

Within every family, 100% data improves on 25% by approximately 0.005–0.013 nats, but intermediate fractions are not consistently ordered. This is a small observed fraction effect, rather than evidence that data quantity is irrelevant. The repeated full-data trials do not test the reliability of these fraction differences.

4.2 Exposure and descriptive compute

All 262,432 original logged rows satisfy the completed-step target formula. Fifteen runs process at least one cache-equivalent; only XL/75–100% and XXL/50–100% remain below one. Small/25% reaches 8.206 equivalents, ruling out insufficient time to revisit data as an explanation for the entire grid. Replacement sampling still leaves unique coverage unknown.

Full-data exposure declines from 2.077 equivalents for Small to 0.303 for XXL (Table 2). At observed rates, one equivalent would require about 29 minutes for Small and 3 hours 18 minutes for XXL. These are projections, not completed longer runs or guaranteed epochs. Equal exposure is a different experimental control from equal time.

The full-data $6ND$ proxy increases from approximately 9.50×10^{16} to 2.18×10^{17} despite equal clocks. A descriptive quadratic fitted in log-parameter count to the five rounded full-data endpoints has a vertex near 130M parameters (Figure 2). Five single-seed observations with unequal validation samples cannot establish a scaling law or optimal size.

4.3 Budget and context extensions

Among the three 30-minute trials, Large has the lowest sampled CE (1.5803), ahead of XL (1.6332) and XXL (1.6797); the one-hour full-data screen favours XL (Table 3). This suggests budget sensitivity, but changed hardware labels and unequal validation workloads prevent attributing the ranking change solely to time.

The context-512 XL recipe records validation CE 1.4555, versus 1.5068 for the original context-256 XL, and sampled-test CE/PPL 1.4606/4.31. It processes 190.11M targets versus 279.83M. This promising variant also changes target batch, position embeddings, assessment windows, and recorded allocation. It therefore does not isolate the effect of context length. Its sampled-test PPL cannot be ranked directly against the original full-test results.

4.4 Historical tests and generated stories

All 20 original test CE/PPL/BPB exports appear in Table 6. Full-data test CE/PPL are Small 1.833995/6.2588, Medium 1.641483/5.1628, Large 1.539026/4.6601, XL 1.482373/4.4034, and XXL 1.517420/4.5604. The original validation-selected XL/50% has 1.484568/4.4131. These historical results agree with the broad family pattern, but all were inspected; they are not untouched confirmation for later choices. No fresh test export is supplied for the six repeated trials.

The saved continuations also reveal repetition and inconsistent characters. For "Tom had a big red ball.", Small/full switches to an unintroduced Tim; XL/full later has a girl tell Tom "I want to pet you"; XXL/full has Sam say "I'm Sam. I'm Tom." These illustrate errors that average loss does not summarize. Five generic prompts and one unblinded sampling seed cannot estimate error rates or establish a qualitative family ranking.

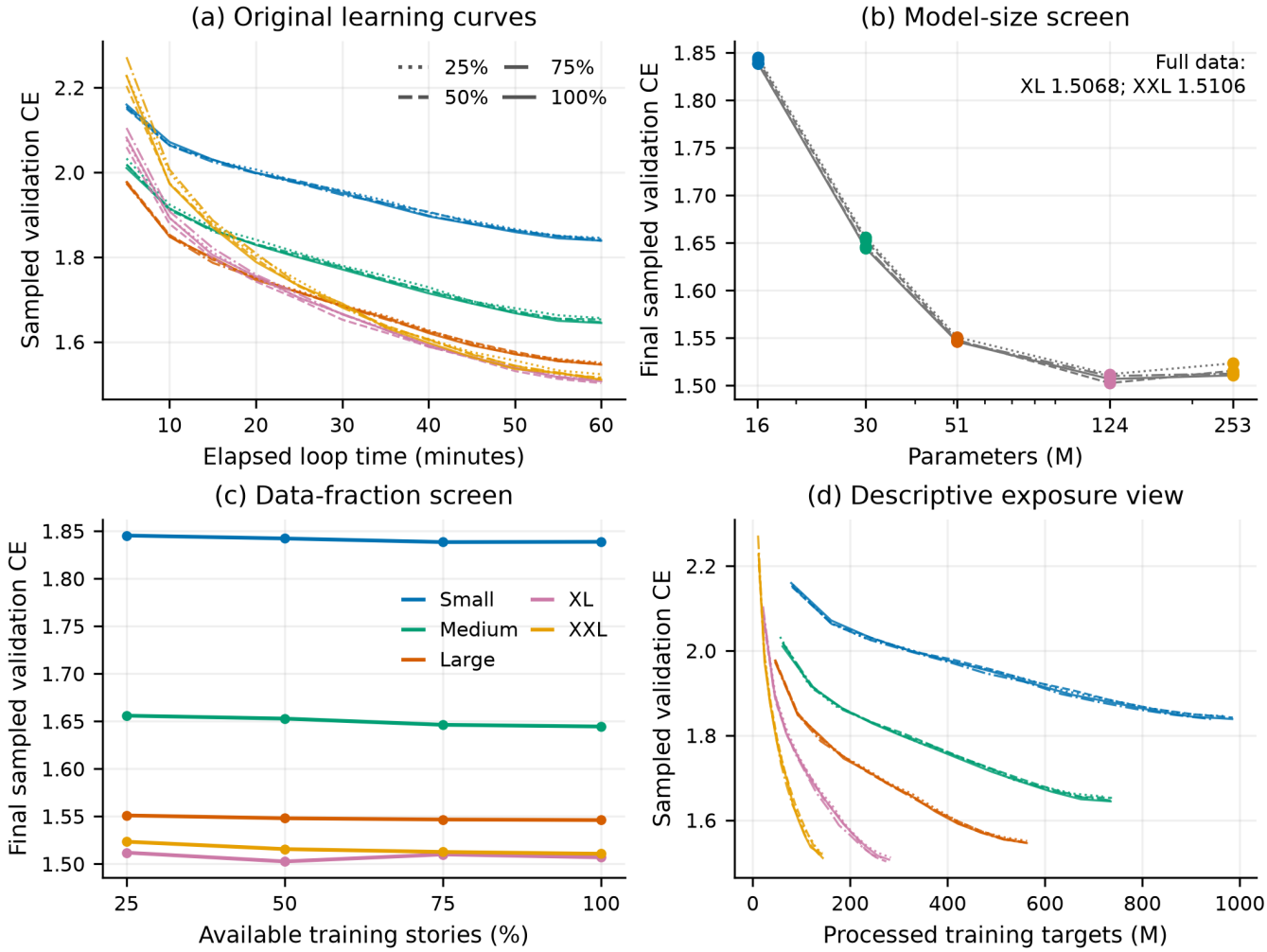


Figure 1: Original 20-run screen: validation against time, final loss against size, data-fraction effects, and loss against training exposure. Colours identify families and line styles identify story fractions. The flattening near XL/XXL motivates focused re-testing. Validation workloads differ, and equal targets do not align the time-based learning-rate schedule.

Table 2: Original full-data exposure relative to the 473,992,236-token cache. Rates include timed validation/logging; projected equivalent times (h:mm:ss) assume stable throughput and do not certify unique coverage. The $6ND$ column estimates training compute, without measuring device utilization.

Family	Targets (M)	Rate (k/s)	Cache equivalents	Time / equivalent	$6ND$ (10^{17})
Small	984.326	273.4	2.077	0:28:54	0.950
Medium	734.642	204.1	1.550	0:38:43	1.323
Large	562.430	156.2	1.187	0:50:34	1.722
XL	279.831	77.7	0.590	1:41:38	2.078
XXL	143.655	39.9	0.303	3:17:59	2.181

4.5 Why XL and XXL were re-tested

XL is the strongest family in the initial screen. XXL has approximately 2.05 times its parameters but only 51.3% of its full-data exposure. Their sampled CE gap is just 0.003782, compared with 0.039197 between Large and XL. Figure 1 therefore identifies a close, practically relevant comparison whose capacity/exposure trade-off remains unresolved. Unequal validation samples motivate aligned scoring first; a single seed separately motivates fresh training.

Common scoring retains XL's lead (Table 4). The full-data XXL-minus-XL gap becomes 0.034285. XL/full also beats XL/50% by 0.001798, reversing the original sampled ordering. This small change does not establish a fraction optimum. Coverage, window boundaries, and intended precision change together, so the larger finalist gap cannot be attributed solely to sample size.

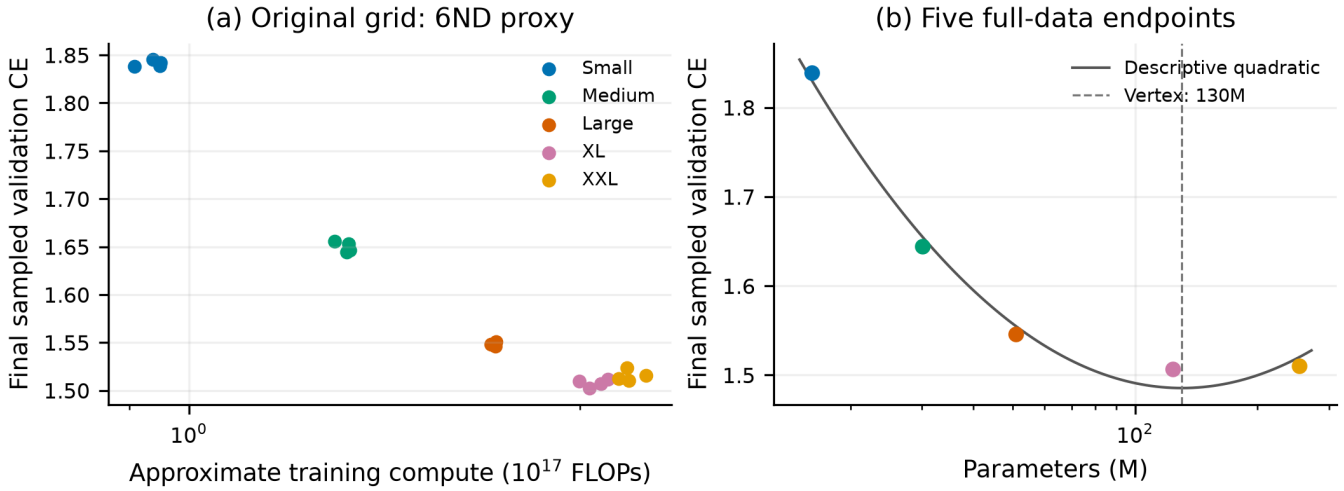


Figure 2: Original endpoints against the approximate $6ND$ compute proxy (left) and a quadratic fit to the five full-data endpoints (right). The approximately 130M vertex describes this screen; actual device compute and utilization were not measured.

Table 3: Budget/context extensions and one-hour full-data references (seed 42, dropout 0.1, bfloat16). Hardware labels differ: H100 NVL versus MIG 3g.47gb. Test values are rounded sampled exports with an unsaved exact protocol; dashes indicate no corresponding sampled export.

Run	Minutes	Context	Targets/update	Targets (M)	Val. CE	Test CE	Test PPL	Test targets
large_t1800	30.0	256	4,096	366.367	1.580347	1.5504	4.71	65,536
xl_t1800	30.0	256	4,096	110.592	1.633249	1.5942	4.92	32,768
xxl_t1800	30.0	256	4,096	57.639	1.679670	1.6987	5.47	16,384
xl_ctx512	60.0	512	8,192	190.112	1.455457	1.4606	4.31	32,768
Original full-data one-hour references (different recorded MIG allocation):								
large_data100	60.0	256	4,096	562.430	1.546009	—	—	—
xl_data100	60.0	256	4,096	279.831	1.506811	—	—	—
xxl_data100	60.0	256	4,096	143.655	1.510593	—	—	—

Table 4: Four archived checkpoints re-scored on a reported 3,808,643 common validation targets. Sampled and common CE use different assessment protocols.

Checkpoint	Sampled CE	Common CE	Common PPL
Large / 100%	1.546009	1.546940	4.6971
XL / 50%	1.502522	1.491074	4.4419
XL / 100%	1.506811	1.489276	4.4339
XXL / 100%	1.510593	1.523562	4.5885

4.6 Three-seed repetitions and throughput

XL's mean reported full-validation CE is 1.494657 (sample SD 0.003793), versus 1.524387 (SD 0.009792) for XXL. Paired XXL-minus-XL gaps are 0.039417, 0.034309, and 0.015466: all three pairs favour XL, as does every observed cross-family comparison (Figure 3). These comparisons reuse six training runs; they are not additional independent replications.

Geometric-mean PPL is 4.457805 for XL and 4.592329 for XXL, approximately 2.93% lower for XL. The mean paired CE gap is 0.029731. An illustrative paired t 95% interval is $[-0.001606, 0.061068]$; it includes zero and relies on a normality assumption that three pairs cannot reliably assess. We report an observed repeatable direction, without a significance claim.

All 29,961 supplied repetition rows have finite recorded losses and consistent counters; loop times span 3,600.008–3,600.101 seconds. Recorded controls, code/cache hashes, software, hardware label, and precision match. XL processes 268.939M targets on average versus 139.278M for XXL, a 1.931-fold ratio (74,704 versus 38,688 targets/second). XXL seed 44 processes 9.8% more targets than seed 43 and has the best XXL loss. Observed variability therefore combines training randomness with execution/exposure variation.

At 120M targets, interpolation of sampled curves gives CE about 1.676–1.678 for XL and 1.545–1.551 for XXL (Figure 4). XXL reaches that exposure later in its time-based schedule. These curves suggest a capacity/ exposure trade-off but do not isolate its cause. All six improve in the final five minutes, without establishing

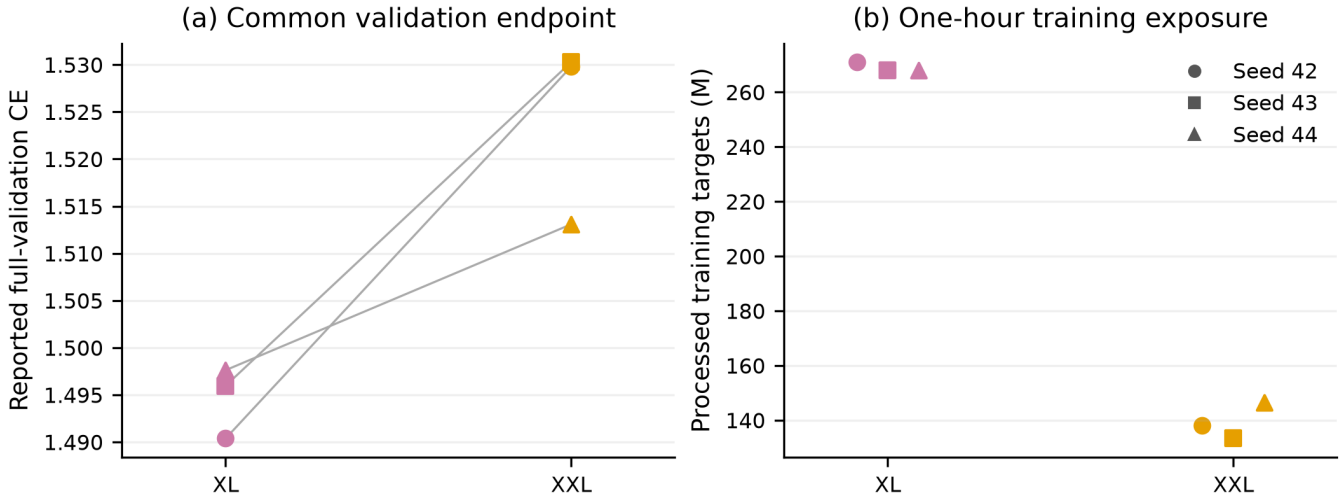


Figure 3: Returned common-validation scores and processed training targets for six fresh one-hour trials. Lines join matching seed labels, not aligned initialization states. All three pairs favour XL. Complete individual results are retained in Table 5.

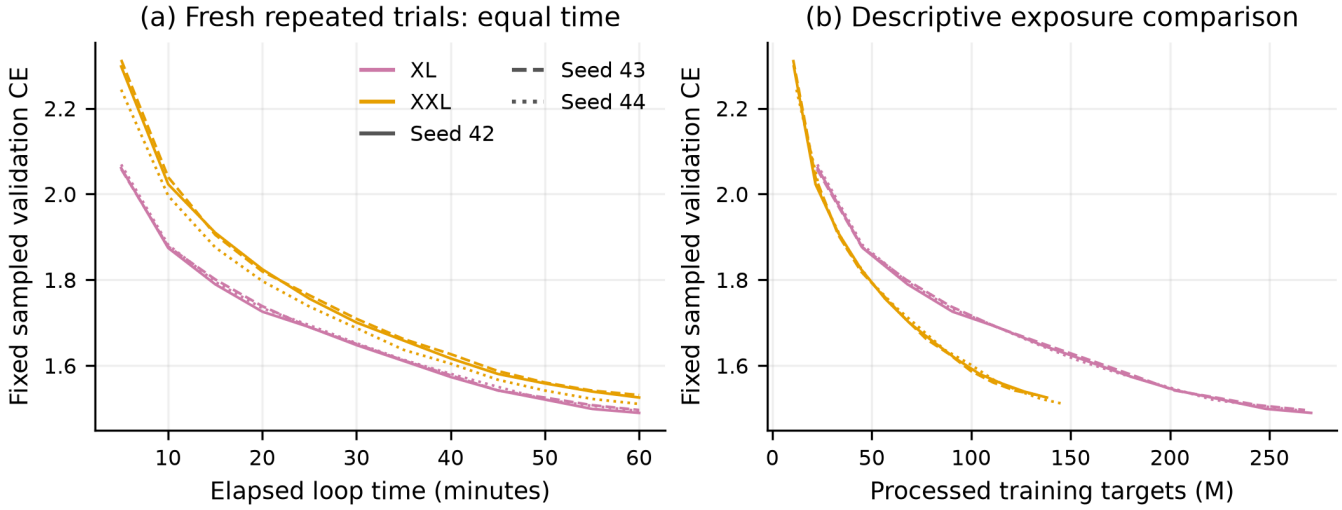


Figure 4: Fresh sampled-validation curves against time and processed targets. XL leads after one hour; XXL appears better at comparable exposure. Time-based schedules differ at equal targets, so this is descriptive evidence. Sampled curve values are distinct from the full-validation scores in Figure 3.

convergence or eventual XXL overtaking.

5 Discussion and limitations

The screen, common-scoring audit, and fresh trials consistently identify XL as a useful one-hour configuration under the shared recipe. Equal time deliberately allows unequal exposure: this answers a practical resource-allocation question, while leaving intrinsic capacity effects unresolved. Depth, width, and heads also change together, and no per-family optimizer search was performed; larger models are also more sensitive to the learning rate [12]. The 2.9% PPL improvement has no measured translation into perceived story quality.

Data-fraction effects are smaller and remain single-seed observations. Sufficient data, corpus redundancy, and optimization limits are plausible explanations that the study does not distinguish; repeating data for a few epochs is reported to cost little [10]. The budget/context extensions raise useful questions but change multiple conditions. Controlled follow-ups would match allocation, target batch, and assessment before isolating context or time effects.

The targeted repetitions have only three seeds per family and were chosen after exploratory inspection. Their SD includes throughput variation. TinyStories is a restricted synthetic domain, and neither loss nor anecdotal stories establish broad language ability. Blind ratings and balanced context diagnostics have no completed results.

Some reproducibility gaps remain. The dataset revi-

sion is unpinned, historical cache ordering is not fully verified, and duplication was not audited. New hashes establish cache agreement among repetitions, not identity with every old cache. The six repeated-run checkpoints and timing summaries are on the checkpoint repository, and the follow-up configs, offline score tables, and scoring script are in the project repository (`configs/followups/`, `analysis/followups/`, `score_checkpoints.py`). Returned counts match the intended validation endpoint; some per-story bootstrap intervals are summarised rather than exported, so conditional intervals cannot be fully re-derived from the aggregate rows. The four archived checkpoint identifiers match the downloaded-baseline manifest. All historical tests were already inspected.

We then ran the prepared equal-target pair: 35,000 updates (143,360,000 targets) with matched step-based warmup/cosine schedules for fresh XL and XXL models, seed 42, on the recorded MIG allocation. At equal targets the ranking reverses: XXL reaches full-validation CE 1.5209 (PPL 4.5763) against 1.5743 (PPL 4.8272) for XL, a 0.0534-nat advantage for XXL, while needing about 62 minutes against 31 for XL. This pair is budgeted in updates, not in time, so the XXL run exceeds the one-hour limit of the main comparison by design. This addresses the schedule mismatch at equal exposure and indicates that XL's one-hour lead reflects exposure under the time budget rather than a capacity ceiling; it is a single seeded pair. A single reduced-rate probe (XXL at half the peak learning rate, 1.5×10^{-4}, one hour) did not help: full-validation CE 1.5459 against 1.5298 at the default, and a single value is not a per-size learning-rate search. Four-hour extensions and capability checks remain pending.

6 Conclusion

Across 33 research training runs, the initial screen shows strong improvements through XL and small effects across the tested data fractions. Exposure audits explain how the fixed clock gives different amounts of training text; budget and context extensions identify promising but confounded variants. Common validation and three fresh trials per finalist preserve an observed XL advantage: CE 1.4947 versus 1.5244, with about 2.9% lower PPL and 1.93 times the training exposure. This supports the tested one-hour recipe, with limited replication and provenance. It does not establish a universal optimal size. The matched equal-target pair, however, points to exposure rather than capacity as the driver of the one-hour ranking: at 143M targets XXL leads XL by 0.053 nats (CE 1.5209 versus 1.5743), reversing it under matched schedules. A single seeded pair cannot make this conclusive.

References

[1] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent english? *arXiv preprint arXiv:2305.07759*, 2023. URL https://arxiv.org/abs/2305.07759.

[2] Jonas Geiping and Tom Goldstein. Cramming: Training a language model on a single GPU in one day. In *International Conference on Machine Learning (ICML)*, 2023.

[3] Sean Goedecke. What's the strongest AI model you can train on a laptop in five minutes? Blog post, https://seangoedecke.com/model-on-a-mbp/, 2025.

[4] Alexander Hägele, Elie Bakouch, Atli Kosson, et al. Scaling laws and compute-optimal training beyond fixed training durations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[5] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022. URL https://arxiv.org/abs/2203.15556.

[6] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, et al. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[7] Andrej Karpathy. nanoGPT. Software repository, n.d. URL https://github.com/karpathy/nanoGPT/tree/3adf61e154c3fe3fca428ad6bc3818b27a3b8291. MIT license. Source commit 3adf61e154c3fe3fca428ad6bc3818b27a3b8291.

[8] Mengtian Li, Ersin Yumer, and Deva Ramanan. Budgeted training: Rethinking deep neural network training under resource constraints. In *International Conference on Learning Representations (ICLR)*, 2020.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[10] Niklas Muennighoff, Alexander M. Rush, Boaz Barak, et al. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information*

Processing Systems, volume 30, 2017. URL https://arxiv.org/abs/1706.03762.

[12] Mitchell Wortsman, Peter J. Liu, Lechao Xiao, Katie Everett, et al. Small-scale proxies for large-scale Transformer training instabilities. In *International Conference on Learning Representations (ICLR)*, 2024.

A Complete results

Research source records are `results/<run>/`, the four extension directories under `runs/`, and `runs/followups_1h/`. Score exports and the archived checkpoint manifest are under `analysis/followups/`; raw configuration and CSV files preserve full settings. Repetition logs record project commit `0052a4b3add7` (full identifier in configurations) and matching code/cache hashes. Pipeline checks cover shifted targets, causal masking, tied weights, tiny-batch overfitting, checkpoint loading, generation, and counters. Pilot/development scores in Table 7 document execution, not controlled scientific comparisons.

Code and archived weights are available in the project repository and checkpoint repository. This revision adds three follow-up training runs (the matched-target XL/XXL pair and a reduced-rate XXL run); no earlier checkpoints were retrained. Checkpoints of the four budget/context extensions were not retained; their logs and configurations are in `runs/`.

Table 5: All six fresh one-hour repetitions. Targets count completed training predictions in millions; CE and PPL use the reported common-validation endpoint of 3,808,643 targets.

Family	Seed	Steps	Targets (M)	Loop seconds	CE	PPL
XL	42	66,166	271.016	3600.047	1.490381	4.4388
XL	43	65,408	267.911	3600.057	1.495970	4.4637
XL	44	65,403	267.891	3600.020	1.497618	4.4710
XXL	42	33,721	138.121	3600.033	1.529798	4.6172
XXL	43	32,550	133.325	3600.101	1.530280	4.6195
XXL	44	35,739	146.387	3600.008	1.513084	4.5407

Table 6: Complete 20-run screen (seed 42, one hour; architectures in Table 1). Sampled validation workloads differ by family; cache-equivalents do not certify unique-data epochs. All historical test exports score 957,184 targets and 3,914,357 decoded bytes; these test results have already been inspected.

Family	Stories (%)	Training targets (M)	Cache equivalents	Sampled val. CE	Sampled val. PPL	Full-test CE	Full-test PPL	Full-test BPB
Small	25	971.956	8.206	1.845226	6.3295	1.842242	6.3107	0.649915
Small	50	985.485	4.159	1.842098	6.3098	1.839643	6.2943	0.648997
Small	75	940.806	2.647	1.838314	6.2859	1.837839	6.2829	0.648361
Small	100	984.326	2.077	1.838635	6.2880	1.833995	6.2588	0.647005
Medium	25	719.073	6.071	1.655840	5.2375	1.650172	5.2079	0.582155
Medium	50	737.329	3.111	1.652656	5.2208	1.648475	5.1990	0.581556
Medium	75	738.320	2.078	1.646227	5.1874	1.643048	5.1709	0.579642
Medium	100	734.642	1.550	1.644364	5.1777	1.641483	5.1628	0.579090
Large	25	563.151	4.754	1.550899	4.7157	1.547532	4.6999	0.545945
Large	50	557.892	2.354	1.547867	4.7014	1.544506	4.6857	0.544878
Large	75	562.012	1.581	1.546585	4.6954	1.540769	4.6682	0.543560
Large	100	562.430	1.187	1.546009	4.6927	1.539026	4.6601	0.542945
XL	25	283.300	2.392	1.511787	4.5348	1.491843	4.4453	0.526299
XL	50	273.949	1.156	1.502522	4.4930	1.484568	4.4131	0.523732
XL	75	269.087	0.757	1.509920	4.5264	1.489011	4.4327	0.525300
XL	100	279.831	0.590	1.506811	4.5123	1.482373	4.4034	0.522958
XXL	25	143.176	1.209	1.523338	4.5875	1.529087	4.6140	0.539438
XXL	50	148.185	0.625	1.515496	4.5517	1.518175	4.5639	0.535589
XXL	75	141.128	0.397	1.512485	4.5380	1.523112	4.5865	0.537330
XXL	100	143.655	0.303	1.510593	4.5294	1.517420	4.5604	0.535322

Table 7: Nine retained pilot/development runs under differing settings. $L/d/H/T$ denotes layers/width/heads/context. Seconds measure loop time, targets count predictions, and CE is sampled validation. These are pipeline checks rather than controlled family comparisons.

Run	$L/d/H/T$	Device	Precision	Seconds	Targets	Final CE
`pilot`	4/256/4/256	H100 MIG	bf16	120.010	31,834,112	2.421298
`fresh-prep-check`	1/64/4/32	MPS	fp32	0.421	64	10.794510
`config-smoke`	2/128/4/64	MPS	fp32	3.004	26,624	8.143094
`config-width-check`	2/64/4/64	MPS	fp32	0.240	256	10.816594
`five-minute`	4/256/4/256	MPS	fp32	300.598	1,568,768	4.183026
`smoke`	2/128/4/64	MPS	fp32	30.008	142,080	5.147076
`smoke-default`	4/256/4/256	MPS	fp32	1.840	4,096	10.629810
`smoke-periodic`	1/64/4/32	MPS	fp32	2.036	2,048	10.819857
`width512`	4/512/4/256	MPS	fp32	360.455	958,464	3.990984

Pilot: 25% training pool; width512: full cache. Fresh-prep-check uses 50/20/20 training/validation/test stories; other local runs reference 2,000/200/200, although some legacy configurations record only cache paths. Local dropout is 0; pilot dropout is 0.1. MPS is Apple's GPU backend. Records are under `runs/pilot/` and the extension directories under `runs/`. Finishing an update can exceed very short time limits.